

Assignment 3: Profile Card UI

Project Title: Build a Flutter profile card screen using Column, Row, Container, CircleAvatar, Text, and Icon widgets with custom theme colours.

Student Name: Sudarshan Devadiga

Roll No.: 150096724089

Github: https://github.com/SudarshanDevadiga/Profile_Card_UI.git

1. Project Objective

The objective of this project is to design and implement a modern, modular, and fully responsive user profile interface using the Flutter framework. The application demonstrates proficiency in UI layout structures, state management (via StatefulWidget), interactive components, dynamic theming (Dark/Light mode), and implicit animations.

2. Technical Stack

- **Framework:** Flutter SDK
- **Language:** Dart
- **Core Libraries:** material.dart
- **Version Control:** Git/GitHub

3. Key Features & Implementation Details

3.1. Responsive Layout Architecture

- The application utilizes a LayoutBuilder to dynamically adapt to different screen dimensions.
- **Mobile View:** Uses a vertical Column structure for a stacked layout ensuring readability on smaller screens.
- **Tablet/Desktop View:** Uses a horizontal Row structure with flex properties (Expanded) to present information side-by-side, maximizing screen real estate.
- Padding and spacing dynamically adjust based on device width constraints.

3.2. Dynamic Theming (Light and Dark Mode)

- The application implements global state management at the root level (ProfileApp) to toggle between Light and Dark themes. (Customized to Slate Light and Slate Dark for this project's Cloud-Native aesthetic).
- ThemeData is configured for both brightness: Brightness.light and brightness: Brightness.dark.
- A callback function (onThemeToggle) is passed down to the ProfileScreen widget, allowing a user-triggered state change that dynamically re-renders all text, container, and border colors across the application without requiring a full app restart.

3.3. Staggered Entrance Animations & Visual Elements

- An AnimationController paired with SingleTickerProviderStateMixin is used to create a premium, staggered cascade effect upon screen load.
- **Fade & Slide Transitions:** Elements utilize Tween animations with Interval curves to load sequentially, drawing the user's eye down the page.
- **Live Status Indicator:** Integrates a continuous, looping pulse animation using `ScaleTransition` on a status dot to indicate system availability.
- **Dynamic Avatar:** Features a circular avatar with customized borders and an accent-colored drop shadow for a modern aesthetic.

3.4. Interactive State Management (Connect Button)

- The "Connect" button is separated into an isolated StatefulWidget to manage local state independently from the parent screen.
- **Interaction Physics:** Utilizes GestureDetector paired with AnimatedScale to create a shrinking physical press effect on user interaction.
- **Visual Feedback:** Triggers an AnimatedContainer transition to smoothly morph the button's background and border colors when the state changes.
- **System Notification:** Implements ScaffoldMessenger to trigger a SnackBar pop-up confirming the connection action to the user.

4. Code Structure

To maintain best practices and modularity, the codebase is strictly separated into two primary files within the repository:

1. **lib/main.dart:** Acts as the application entry point. It contains the root MaterialApp, manages the global theme state, and establishes the base color palettes.

Dart

```
import 'package:flutter/material.dart';
import 'profile_screen.dart';

void main() {
  runApp(const ProfileApp());
}

class ProfileApp extends StatefulWidget {
  const ProfileApp({super.key});

  @override
  State<ProfileApp> createState() => _ProfileAppState();
}

class _ProfileAppState extends State<ProfileApp> {
  ThemeMode _themeMode = ThemeMode.light;

  void _toggleTheme() {
```

```

    setState() {
      themeMode = _themeMode == ThemeMode.light ? ThemeMode.dark : ThemeMode.light;
    });
  }

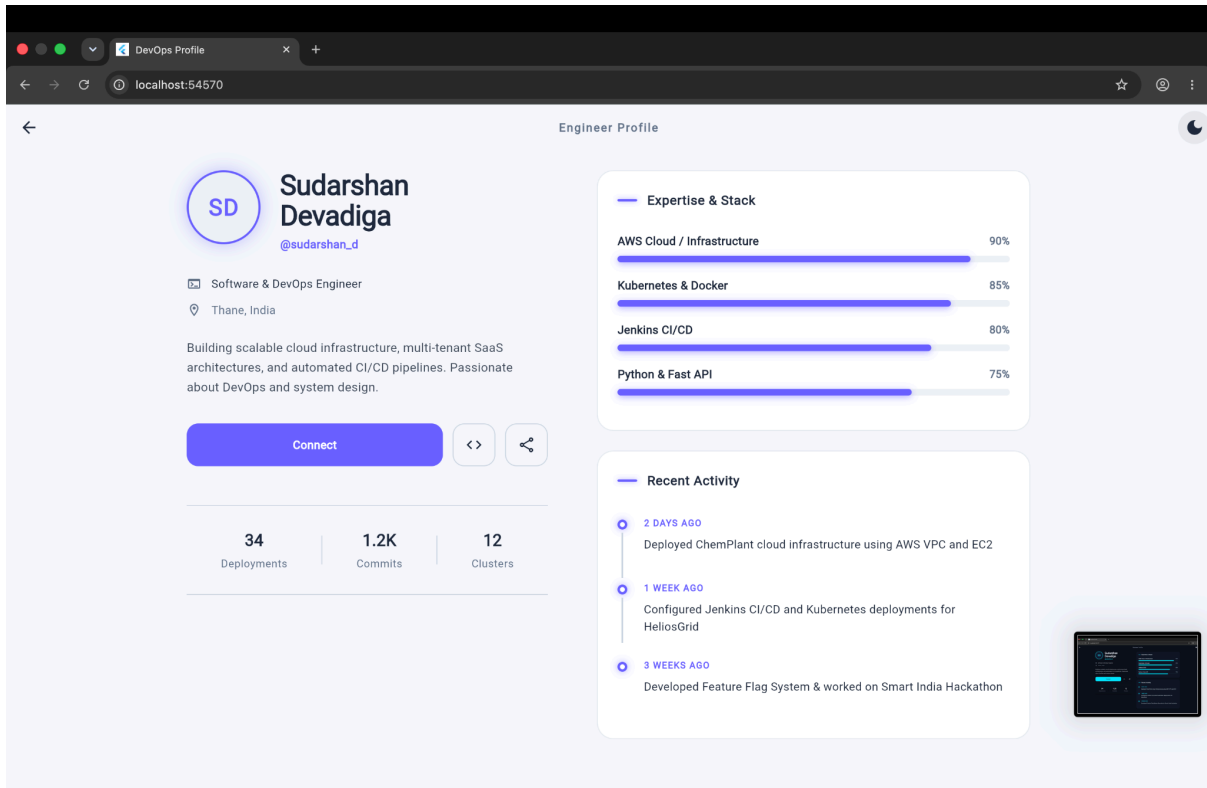
  @override
  Widget build(BuildContext context) {
    return MaterialApp(
      debugShowCheckedModeBanner: false,
      title: 'DevOps Profile',
      themeMode: _themeMode,
      theme: ThemeData(
        brightness: Brightness.light,
        fontFamily: 'Roboto',
        scaffoldBackgroundColor: const Color(0xFF1F5F9),
      ),
      darkTheme: ThemeData(
        brightness: Brightness.dark,
        fontFamily: 'Roboto',
        scaffoldBackgroundColor: const Color(0xFF0F172A),
      ),
      home: ProfileScreen(
        onThemeToggle: _toggleTheme,
        isDarkMode: _themeMode == ThemeMode.dark,
      ),
    );
  }
}

```

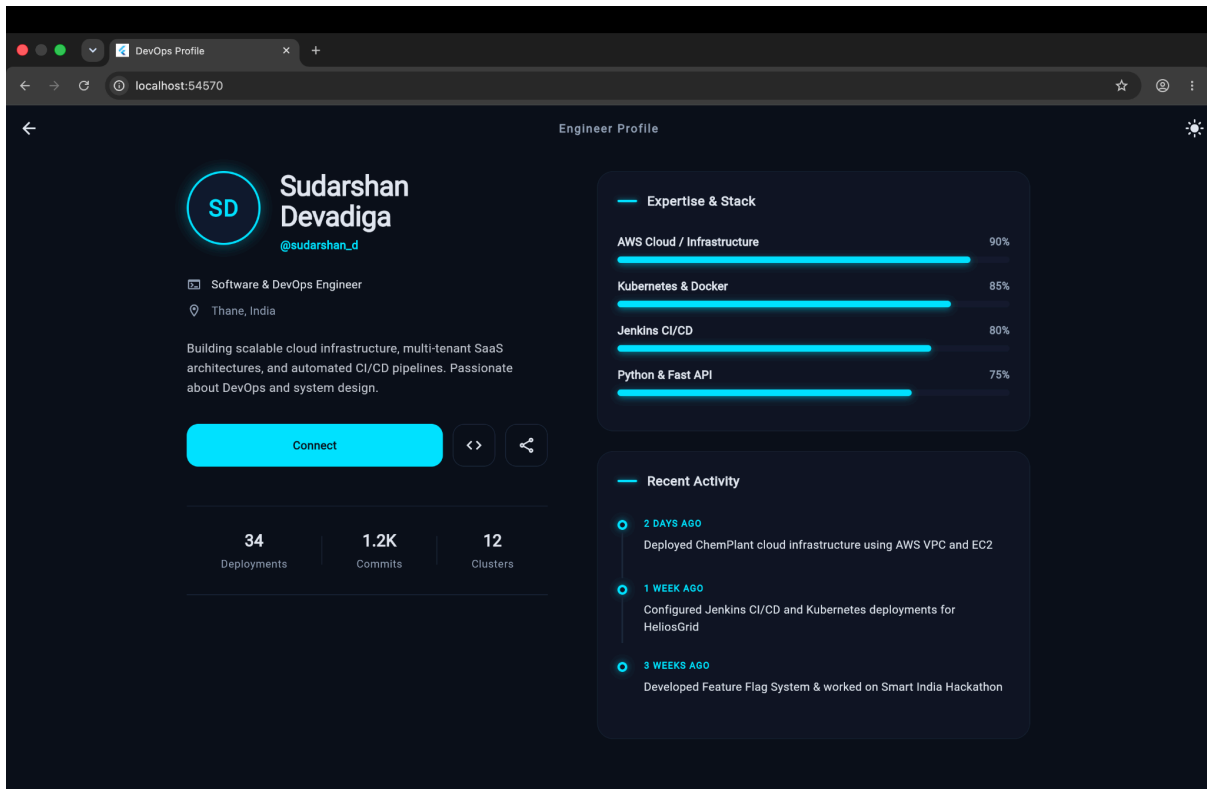
2. lib/profile_screen.dart: Houses the core UI logic, responsive layout builders, animation controllers, and all abstracted helper widgets (e.g., timeline builders, stat counters, and the interactive follow button).

5. Application Output & Screenshots

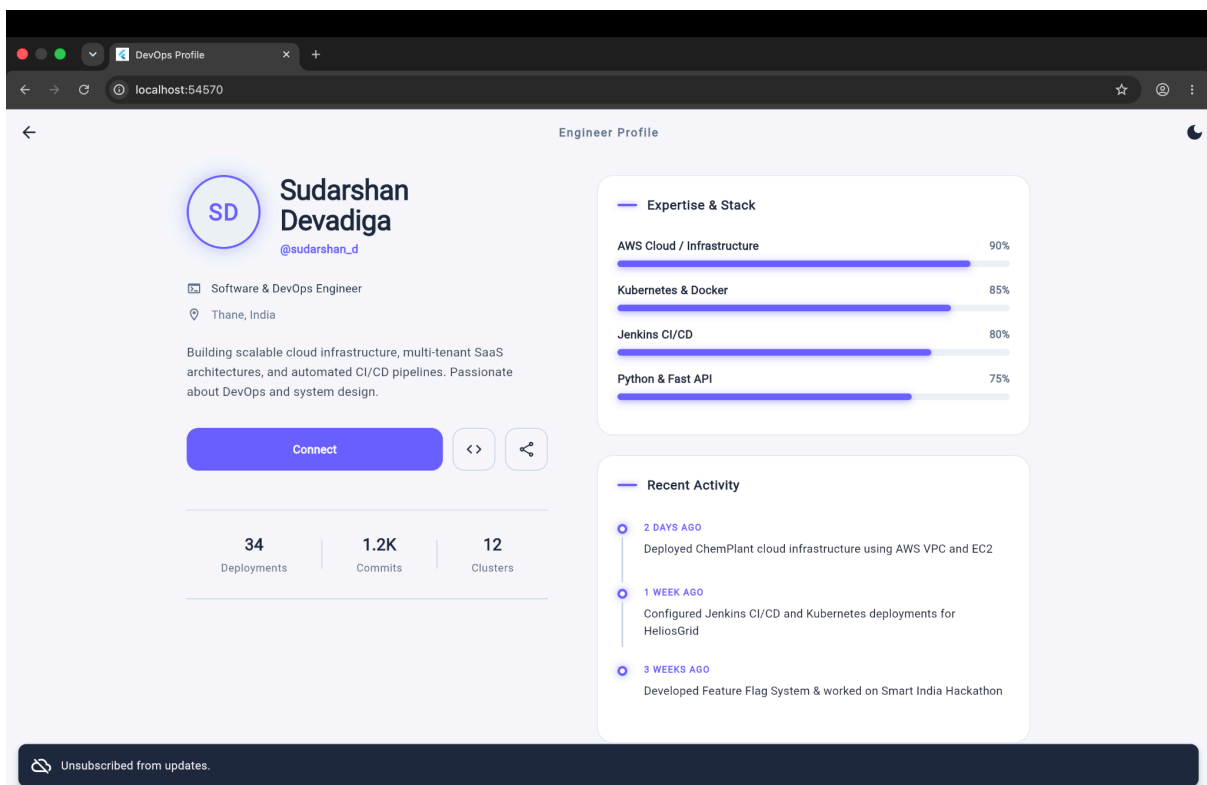
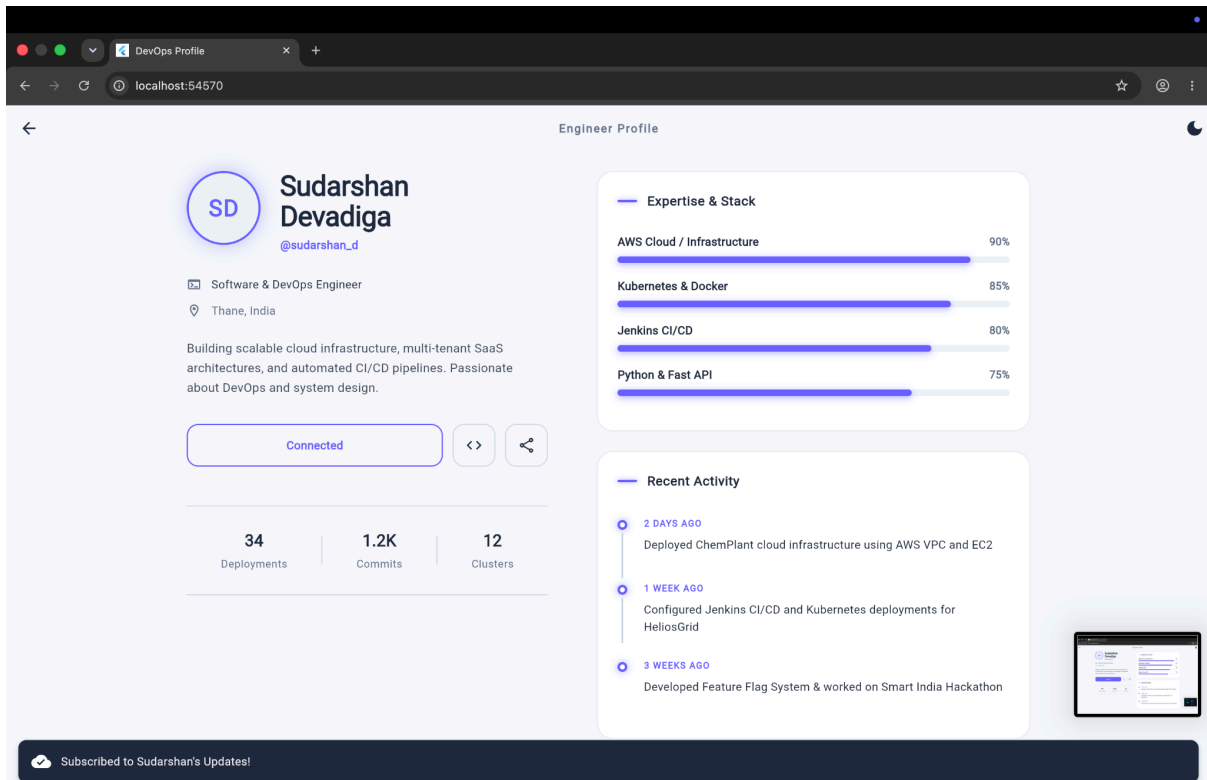
- 5.1. Light Mode Interface:



- 5.2. Dark Mode Interface:



- **5.3. Interactive State & SnackBar Notification:**



6. Learning Outcomes

This project successfully validated core mobile application development competencies, specifically:

- Decomposing complex UIs into reusable, modular, and private helper methods.
- Managing both global application state (theming) and local component state (button interactions) effectively.
- Enhancing user experience (UX) through implicit transitions, touch-feedback scaling, and staggered timeline animations.
- Building constraint-aware interfaces that function seamlessly across varying form factors.
- Utilizing Git and GitHub for version control and project hosting.